

Grading Fire Risk Before the Alarm: A Fog-to-Cloud Wildfire Monitor for a Two-Station Ranger Network

Deekonda Rakshan
Student ID: X25180754
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25180754@student.ncirl.ie

Abstract—A forest fire announces itself long before it is a fire. The air warms, the humidity falls, smoke begins to thicken, the wind rises, and the soil dries out, and a ranger who can watch those five signals together has a warning that a single confirmed alarm, raised only once smoke is already dense, cannot give. This project is built around exactly that difference. Ten sensor processes drive five environmental signals, temperature, humidity, smoke density, wind speed, and soil moisture, across two ranger stations; a fog node on the edge host windows each station-and-signal stream, reduces it to a summary, and raises the hard hazard alarms locally, so a confirmed danger is named in the window it appears rather than after a trip to a distant region. Behind the edge, a serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists every window, and a dashboard served through Amazon API Gateway does the one thing the hard alarms cannot: from the same stored windows, but against earlier and lower thresholds, it computes a graded fire-risk index from zero to four for each station and shows it as a dial that climbs gradually as conditions worsen, giving an escalation gradient before any hard alarm fires. The index is never stored; it is derived fresh on every read. Each tier is justified against the alternatives it displaces, and the system was provisioned to a real Amazon Web Services account rather than to a local emulator. An automated suite of 95 tests passes, and verification against the live endpoint confirmed both stations streaming all five signals, the risk dial grading each station, and a fire-detected alarm firing on a smoke spike, with the static frontend delivered from Amazon S3.

Keywords—wildfire detection, environmental monitoring, wireless sensor networks, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

Fire is the end of a process, not its beginning. Long before a forest actually ignites, the conditions that make ignition likely accumulate in the open air, and the value of monitoring a forest lies in reading those conditions early rather than in confirming a blaze once it has started. Five signals, read together, describe that build-up: temperature and humidity set how dry the air is, a smoke-density reading is the first physical trace of combustion, wind speed governs how fast a spark would spread, and soil moisture measures how ready the ground fuel is to burn. Wireless-sensor deployments in forests

are the established way to gather exactly these signals across terrain no operator can watch by eye [1], and fog-assisted architectures have been proposed specifically to process that wildfire telemetry close to where it is collected rather than shipping it all to a distant cloud [2].

The distributed-systems question is where that processing happens. The Internet of Things makes dense environmental sensing cheap [3], but streaming every raw sample from every station to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, seldom change a decision, and it defers the one moment that matters, a signal crossing into its hazard range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that is idle most of the time [8].

This project builds that two-tier design for a scaled but faithful slice of a monitored forest: two ranger stations, designated station-1 and station-2, each carrying the same five signals, run as ten independent sensor processes. What makes the design its own is a deliberate split in how danger is expressed. The fog node raises hard alarms at high thresholds, the confirmed hazards a ranger must respond to now, while the dashboard derives a graded fire-risk index from the same signals but against earlier, lower thresholds, so risk is shown as a rising dial well before any hard alarm trips. The two are kept apart on purpose: the hard alarm says a danger is present, and the graded index says a danger is building, and a forest service needs both the certainty of the first and the lead time of the second.

The work pursues four objectives, each shown on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous environmental signals at sampling and dispatch rates that are

configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into an operational reading, in particular the graded fire-risk index that gives early warning ahead of a hard alarm. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-station scale rather than a full sensor network across a region. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one environmental signal at one station. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn uniformly from a small symmetric range and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real field probe rather than independent noise. The step sizes are set per signal to match how the real quantity moves, so smoke density and wind speed, which spike quickly in fire weather, drift faster between ticks than soil moisture, which changes only slowly. Each process runs two independent timing loops, one for sampling and one for dispatch, and each loop is a recursive timer that reschedules itself only at the end of its own body, so a slow dispatch can never let two dispatch calls overlap and the two rates never share a single tick even when they are set equal. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal, the station, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte. The process posts it over HTTP to the fog node's ingest endpoint, and if that post fails because the fog node is briefly unreachable, the batch is retained in the process's own outbox and retried on the next tick rather than dropped, so a transient outage delays readings instead of losing them.

The fog node is a Node.js service built directly on the runtime's own HTTP server with manual path dispatch and no web framework, running as its own container on the edge host distinct from the sensor processes and the cloud functions. Every ingest request is validated before anything is buffered, so a malformed batch is rejected with a clear error rather than corrupting a window. The buffering itself is event-driven, which is the piece of the edge that most repays attention. Rather than have the request handler write directly into a shared structure, the node emits a reading event, and a single listener owns the map of accumulated readings and does all the accumulation in response to that event; ingestion and buffering are decoupled through publish-subscribe so the request handler returns as soon as the event is emitted and only one place ever mutates the buffer. When the window boundary arrives on a fixed timer, the buffer is snapshotted and cleared in one step so a fresh empty window is in place immediately, and the snapshot is summarised outside that step.

When a window closes, each non-empty station-and-signal group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the hard hazard rules. Those rules are a flat list of descriptors, each owning the signal it applies to and a predicate on the window summary, evaluated by a simple filter-and-map with no dispatch table in the way: an average temperature above forty-two degrees raises an extreme-heat alarm, an average smoke density above one hundred and fifty parts per million raises a fire-detected alarm, an average wind speed above sixty kilometres an hour raises a high-wind warning, and an average soil moisture below ten per cent raises a drought-risk alarm. Humidity deliberately carries no hard rule; it is an input to the risk picture rather than a hazard in itself, and is stored and shown but never alarms on its own. These thresholds are set high on purpose, because a hard alarm is a claim that a danger is actually present, and the graded early-warning index described in the serving layer is what covers the space below them.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a whole flush costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. The queue's address is resolved once and memoised behind a cache keyed by queue name so it is not looked up on every publish, retried while the queue is still being provisioned because on a fresh deployment it may not exist when the edge host boots, and a failed send is logged and the cycle continues rather than crashing the node, a bias toward keeping the edge live that suits a monitoring workload where the next window is only seconds away.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's summaries, one function drains the queue and writes to

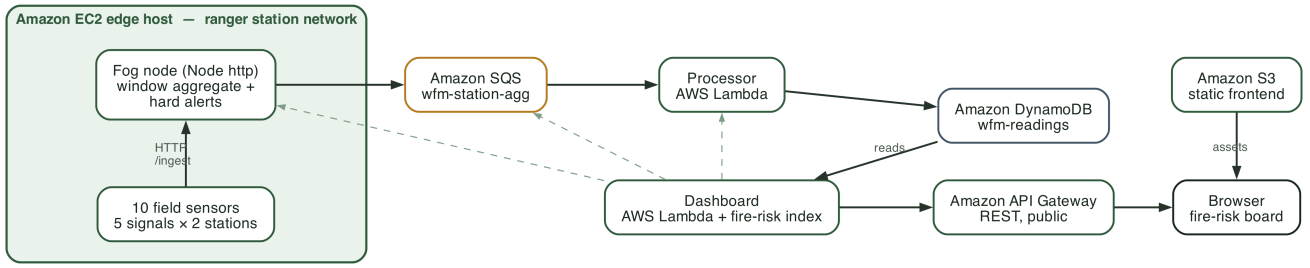


Fig. 1. Deployment topology. Ten sensor processes and the fog node share one Amazon EC2 edge host at the ranger station network. Solid arrows are the ingest path, from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table, and the serving path, from the dashboard function’s read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3. The dashboard derives the graded fire-risk index at read time. The dashed edges are the three further status sources the dashboard polls for its health strip: the fog node’s reachability, the queue’s depth, and the processor function’s deployment state.

storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure propagates toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved up front [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard’s own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12].

The key design follows straight from the query. Each stored window is keyed on its signal as the partition and on a sort value that leads with the window’s closing timestamp and then the station identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the station in the sort tail keeps the two stations’ windows distinct within the same signal.

C. Serving Layer, the Fire-Risk Index, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. Inside the deployed function, incoming requests are dispatched by a chain of responsibility: an ordered list of handlers is walked, and each either claims the request by returning a response or declines by returning nothing so the walk moves on, so there is no separate route table or matcher and each handler owns its own match condition. It exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a stations endpoint giving each station’s latest window per signal together with its derived risk, a thresholds endpoint that forwards the fog node’s published hard rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue for its depth, the processor function for its deployment state, and the fog node for its reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The graded fire-risk index is the piece of interpretation that the hard alarms cannot provide, and it is computed here, on every read, rather than stored. For each station the dashboard takes the latest window of four of the five signals and counts how many sit in their early-warning zone: temperature above a warm threshold, smoke above a light haze, wind above a breeze, and soil moisture below a dry mark. Each condition contributes one point, giving an integer from zero to four that maps onto a named band from safe through to extreme. The thresholds that feed this count are deliberately lower than the fog node’s hard-alarm limits, so the dial climbs gradually as

conditions worsen instead of jumping to its maximum only when a confirmed alarm already fires, and humidity, which carries no hard rule, likewise contributes no point and is shown only as a raw detail. Deriving the index at read time rather than storing it keeps a single source of truth, the stored windows, and means the risk model can be adjusted without any change to what has already been persisted; a station with incomplete telemetry still yields a meaningful, if partial, score rather than no reading at all. On the page, each station is a card showing its risk dial and band alongside the five raw signal tiles, with a smoke-density trend across both stations and a banner that names any hard alarm currently firing; a fixed browser-side timer re-fetches the stations view, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node’s health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services’ own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The sensor processes, the fog node, and both cloud functions are written in JavaScript on the Node.js runtime. The fog node and the dashboard’s local server are built directly on the runtime’s HTTP server with manual URL parsing and hand-written path dispatch, so neither carries a web framework; the AWS SDK for JavaScript carries the queue, table, and function calls, and the frontend renders its trend with a locally vendored charting library so the page has no external dependency at load time. The pieces are kept as small single-purpose modules, the buffering separated from the aggregation and the risk index separated from the storage access, so each is unit-testable on its own without standing up a server or reaching into module-scope state. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 95 tests spread across every component: the bounded-walk sensor and its retain-on-failure dispatch, the event-driven buffer’s accumulate and snapshot, the window aggregation arithmetic, the hard hazard rules, the batched publisher, the processor’s transform into a stored item, and, on the dashboard side, the fire-risk index, the per-station grouping, the chain-of-responsibility dispatch, the health assembly, and the thresholds proxy. They run without any cloud connection by exercising the units

against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The hazard-rule and risk-index tests are pointed deliberately at the boundaries, checking a value one step inside a threshold and one step outside it, so a rule that should fire and an index point that should count are both pinned down, and the two threshold tiers, hard alarm and early-warning contribution, are tested separately so the gap between them is intentional rather than accidental. The publisher tests check that a flush is split into batches no larger than the queue’s per-call limit, the processor tests check the composite sort value that orders windows within a partition, and the fog ingest validation is proven with a real HTTP request against a running local server rather than only a unit test of the validator in isolation, since that request path is what the deployment actually exercises. The suite runs on every push through GitHub Actions, so a change that breaks a threshold, a batch size, a query shape, or the risk count fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue’s own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node’s health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both stations rendered all five signals with their current value, and each station’s risk dial graded its recent conditions into a band, so a station sitting at an elevated score and a station holding at a low one were both visible at a glance; when a station’s smoke density climbed past the hard limit, the fire-detected alarm fired and the banner named it on



Fig. 2. The deployed dashboard reading from the live endpoint. Each ranger station shows its graded fire-risk dial and band alongside its five raw signal tiles; a smoke-density trend spans both stations; the header reports stations tracked, active alerts, and archived records; the banner names any hard alarm currently firing; and the footer strip shows the four pipeline indicators.

the correct station while the other station stayed unflagged. The backend-statistics endpoint showed the archived record count climbing across successive polls rather than a static number, which confirms the ingestion function was draining the queue and writing on an ongoing basis, and reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five environmental signals across two ranger stations into both a confirmed hazard alarm and an early warning that a hazard is building, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each station-and-signal stream and raises the hard alarms locally, so only compact summaries cross the network and a confirmed danger is named in the window it appears. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 95 automated tests guarding its behaviour and continuous integration running them on every change.

Two reflections stand out from building it. The first is that separating the two ways of expressing danger is what makes the monitor useful rather than merely noisy: a single hard alarm answers whether a hazard is present, but a forest service also needs to see a hazard building, and computing a graded index against thresholds set deliberately lower than the alarm

limits turns the same five signals into an escalation gradient that leads the alarm instead of coinciding with it. The second is that deriving that index at read time rather than storing it kept the design honest, because the stored windows stay the single source of truth and the risk model can be retuned, or a station's missing signal handled gracefully, without rewriting anything already persisted, so the interpretation layer can evolve while the record of what was measured does not.

The work has clear limits. It runs at a two-station scale with simulated sensors, its hard limits and risk thresholds are fixed rather than learned from a site's own seasonal baseline or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a real sensor network across a region and real environmental feeds, to move the fixed thresholds to a managed configuration a forest service could tune per site and per season without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a site's slow slide toward its thresholds can be trended rather than only scored on the latest window. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two stations to many without rethinking the tiers.

REFERENCES

- [1] Y. E. Aslan, I. Korpeoglu, and Ö. Ulusoy, "A Framework for Use of Wireless Sensor Networks in Forest Fire Detection and Monitoring," *Computers, Environment and Urban Systems*, vol. 36, no. 6, pp. 614–625, 2012.
- [2] H. Kaur and S. K. Sood, "Fog-Assisted IoT-Enabled Scalable Network Infrastructure for Wildfire Surveillance," *Journal of Network and Computer Applications*, vol. 144, pp. 171–183, 2019.
- [3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [13] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>

[16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].